Microservices

In software engineering, a **microservice** architecture is an architectural pattern that organizes an application into a collection of loosely coupled, fine-grained services that communicate through lightweight protocols. This pattern is characterized by the ability to develop and deploy services independently, improving modularity, scalability, and adaptability. However, it introduces additional complexity, particularly in managing distributed systems and inter-service communication, making the initial implementation more challenging compared to a monolithic architecture.[1]

Definition

There is no single, universally agreed-upon definition of microservices. However, they are generally characterized by a focus on modularity, with each service designed around a specific business capability. These services are loosely coupled, independently deployable, and often developed and scaled separately, enabling greater flexibility and agility in managing complex systems. Microservices architecture is closely associated with principles such as domain-driven design, decentralization of data and governance, and the flexibility to use different technologies for individual services to best meet their requirements.[2][3][4]

Usage

It is common for microservices architectures to be adopted for cloud-native applications, serverless computing, and applications using lightweight container deployment. According to Fowler, because of the large number (when compared to monolithic application implementations) of services, decentralized continuous delivery and DevOps with holistic service monitoring are necessary to effectively develop, maintain, and operate such applications.[5] A consequence of (and rationale for) following this approach is that the individual microservices can be individually scaled. In the monolithic approach, an application supporting three functions would have to be scaled in its entirety even if only one of these functions had a resource constraint.[6] With microservices, only the microservice supporting the function with resource constraints needs to be scaled out, thus providing resource and cost optimization benefits.[7]

Cell-based architecture in microservices

Cell-based architecture is a distributed computing design in which computational resources are organized into self-contained units called cells. Each cell operates independently, handling a subset of requests while maintaining scalability, fault isolation, and availability. [2][8][9]

A cell typically consists of multiple microservices and functions as an autonomous unit. In some implementations, entire sets of microservices are replicated across multiple cells, enabling requests to be rerouted to another operational cell if one experiences a failure. This approach is intended to improve system-wide resilience by limiting the impact of localized failures. [2][8][9]

Some implementations incorporate circuit breakers within and between cells. Within a cell, circuit breakers may be used to mitigate cascading failures among microservices, while inter-cell circuit breakers can isolate failing cells and redirect traffic to those that remain operational. [2][8][9]

Cell-based architecture has been adopted in certain large-scale distributed systems where fault isolation and redundancy are design priorities. Its implementation varies based on system requirements, infrastructure constraints, and specific operational goals. [2][8][9]

History

In 1999, software developer Peter Rodgers had been working on the Dexter research project at Hewlett Packard Labs, whose aim was to make code less brittle and to make large-scale, complex software systems robust to change.[10] Ultimately this path of research led to the development of resource-oriented computing (ROC), a generalized computation abstraction in which REST is a special subset. In 2005, during a presentation at the Web Services Edge conference, Rodgers argued for "REST-services" and stated that "Software components are Micro-Web-Services... Micro-Services are composed using Unix-like pipelines (the Web meets Unix = true loose-coupling). Services can call services (+multiple language run-times). Complex service assemblies are abstracted behind simple URI interfaces. Any service, at any granularity, can be exposed." He described how a well-designed microservices platform "applies the underlying architectural principles of the Web and REST services together with Unix-like scheduling and pipelines to provide radical flexibility and improved simplicity in service-oriented architectures.[11]

Also in 2005, Alistair Cockburn wrote about hexagonal architecture which is a software design pattern that is used along with the microservices. This pattern makes the design of the microservice possible since it isolates in layers the business logic from the auxiliary services needed in order to deploy and run the microservice completely independent from others.

Microservice granularity

Determining the appropriate level of (micro)service granularity in a microservices architecture often requires iterative collaboration between architects and developers. This process involves evaluating user requirements, service responsibilities, and architectural characteristics, such as non-functional requirements. Neal Ford highlights the role of integrator and disintegrator factors in this context. Integrator factors, such as shared transactions or tightly coupled processes, favor combining services, while disintegrator factors, such as fault tolerance or independent scalability, encourage splitting services to meet operational and architectural goals. Additionally, fitness functions, as proposed by Neal Ford, can be used to validate architectural decisions and service granularity by continuously measuring system qualities or behaviors that are critical to stakeholders, ensuring alignment with overall architectural objectives.[12]

In microservices architectures, service granularity influences testing, deployment, performance, and reliability. Very fine-grained microservices are typically easier to test and deploy independently, but they often experience lower performance and reduced overall reliability due to increased interservice communication and more complex service choreography.[13] Coarse-grained services exhibit contrasting characteristics. They generally provide higher robustness and reliability by minimizing communication

overhead and coordination complexity, but they are more challenging to test and deploy because modifications affect a broader functional scope.[13] The appropriate level of service granularity is determined by business drivers. Architectural decisions commonly begin with identifying these drivers and then aligning architectural characteristics such as performance, scalability, reliability, or deployment flexibility to support them.[13]

Mapping microservices to bounded contexts

A bounded context, a fundamental concept in domain-driven design (DDD), defines a specific area within which a domain model is consistent and valid, ensuring clarity and separation of concerns.[12] In microservices architecture, a bounded context often maps to a microservice, but this relationship can vary depending on the design approach. A one-to-one relationship, where each bounded context is implemented as a single microservice, is typically ideal as it maintains clear boundaries, reduces coupling, and enables independent deployment and scaling. However, other mappings may also be appropriate: a one-to-many relationship can arise when a bounded context is divided into multiple microservices to address varying scalability or other operational needs, while a many-to-one relationship may consolidate multiple bounded contexts into a single microservice for simplicity or to minimize operational overhead. The choice of relationship should balance the principles of DDD with the system's business goals, technical constraints, and operational requirements. [14]

Benefits

The benefit of decomposing an application into different smaller services are numerous:

- Modularity: This makes the application easier to understand, develop, test, and become more resilient to architecture erosion.[15] This benefit is often argued in comparison to the complexity of monolithic architectures.[16]
- Scalability: Since microservices are implemented and deployed independently of each other, i.e. they run within independent processes, they can be monitored and scaled independently.[17]
- Integration of heterogeneous and legacy systems: microservices are considered a viable means for modernizing existing monolithic software application.[18][19] There are experience reports of several companies who have successfully replaced parts of their existing software with microservices or are in the process of doing so.[20] The process for software modernization of legacy applications is done using an incremental approach.[21]
- Distributed development: it parallelizes development by enabling small autonomous teams to develop, deploy and scale their respective services independently.[22] It also allows the architecture of an individual service to emerge through continuous refactoring.[23] Microservice-based architectures facilitate continuous integration, continuous delivery and deployment.[24]

Criticism and concerns

The microservices approach is subject to criticism for a number of issues:

- Services form information barriers.[25]

- Inter-service calls over a network have a higher cost in terms of network latency and message processing time than in-process calls within a monolithic service process.[26]
- Testing and deployment can be complicated.[27]
- Moving responsibilities between services is more difficult.[15] It may involve communication between different teams, rewriting the functionality in another language or fitting it into a different infrastructure.[26] However, microservices can be deployed independently from the rest of the application, while teams working on monoliths need to synchronize to deploy together.[21]
- Viewing the size of services as the primary structuring mechanism can lead to too many services when the alternative of internal modularization may lead to a simpler design. This requires understanding the overall architecture of the applications and interdependencies between components.[28]
- Two-phased commits are regarded as an anti-pattern in microservices-based architectures, resulting in a tighter coupling of all the participants within the transaction. However, the lack of this technology causes awkward dances which have to be implemented by all the transaction participants in order to maintain data consistency.[29]
- Development and support of many services are more challenging if they are built with different tools and technologies - this is especially a problem if engineers move between projects frequently.[30]
- The protocol typically used with microservices (HTTP) was designed for public-facing services, and as such is unsuitable for working internal microservices that often must be impeccably reliable.[31]
- While not specific to microservices, the decomposition methodology often uses functional decomposition, which does not handle changes in the requirements while still adding the complexity of services.[31]
- The very concept of microservice is misleading since there are only services. There is no sound definition of when a service starts or stops being a microservice.[31]
- Data aggregation. In order to have a full view of a working system, it is required to extract data sets from the microservices repositories and aggregate them into a single schema. For example, to be able to create operational reports that are not possible using a single microservice repository.

Complexities

The architecture introduces additional complexity and new problems to deal with, such as latency, message format design,[32] backup/availability/consistency (BAC),[33] load balancing and fault tolerance.[34] All of these problems have to be addressed at scale. The complexity of a monolithic application does not disappear if it is re-implemented as a set of microservices. Some of the complexity gets translated into operational complexity.[35] Other places where the complexity manifests itself are increased network traffic and slower performance. Also, an application made up of any number of microservices has a larger number of interface points to access its respective ecosystem, which increases the architectural complexity.[36] Various organizing principles (such as hypermedia as the engine of application state (HATEOAS), interface and data model documentation captured via Swagger, etc.) have been applied to reduce the impact of such additional complexity.

Antipatterns

- The "data-driven migration antipattern", coined by Mark Richards, highlights the challenges of prioritizing data migration during the transition from a monolithic to a microservices architecture. To address this antipattern, an iterative approach can be helpful where application code is migrated first, with new microservices temporarily relying on the existing monolithic database. Over time, as the system is better understood, data can be decoupled and restructured, enabling individual microservices to operate with their own databases. This strategy can simplify the migration process and reduce data migration errors.[13]
- The "timeout antipattern", coined by Mark Richards, describes the challenges of setting timeout values in distributed systems. Short timeouts may fail legitimate requests prematurely, leading to complex workarounds, while long timeouts can result in slow error responses and poor user experiences. The circuit breaker pattern can address these issues by monitoring service health through mechanisms such as heartbeats, "synthetic transactions", or real-time usage monitoring. This approach can enable faster failure detection and can improve the overall user experience in distributed architectures.[13]
- Reporting on microservices data presents challenges, as retrieving data for a reporting service can either break the bounded contexts of microservices, reduce the timeliness of the data, or both. This applies regardless of whether data is pulled directly from databases, retrieved via HTTP, or collected in batches. Mark Richards refers to this as the "reach-in reporting antipattern".[13] A possible alternative to this approach is for databases to asynchronously push the necessary data to the reporting service instead of the reporting service pulling it. While this method requires a separate contract between microservices and the reporting service and can be complex to implement, it helps preserve bounded contexts while maintaining a high level of data timeliness.[13]

Challenges

Microservices are susceptible to the fallacies of distributed computing – a series of misconceptions that can lead to significant issues in software development and deployment. [12]

Code sharing challenges

Ideally, microservices follow a "share-nothing" architecture. However, in practice, microservices architectures often encounter situations where code must be shared across services. Common approaches to addressing this challenge include utilizing separate shared libraries for reusable components (e.g., a security library), replicating stable modules with minimal changes across services, or, in certain cases, consolidating multiple microservices into a single service to reduce complexity. Each approach has its advantages and trade-offs, depending on the specific context and requirements. [13]

Best practices

Richards & Ford in *Fundamentals of software architecture* (2020) propose each microservice should have its own architectural characteristics (a.k.a. non functional requirements), and architects should not define uniform characteristics for the entire distributed system.[12]

To avoid having to coordinate deployments across different microservices, Sam Newman suggests keeping the interfaces of microservices stable and making backwards-compatible changes as interfaces evolve. On the topic of testing, Newman in *Building Microservices* (2015) proposes consumer-driven contract testing as a better alternative to traditional end-to-end testing in the context of microservices. He also suggests the use of log aggregation and metrics aggregation as well as distributed tracing tools to ensure the observability of systems composed of microservices.[2]

Technologies

Computer microservices can be implemented in different programming languages and might use different infrastructures. Therefore, the most important technology choices are the way microservices communicate with each other (synchronous, asynchronous, UI integration) and the protocols used for the communication (e.g. RESTful HTTP, messaging, GraphQL). In a traditional system, most technology choices like the programming language impact the whole system. Therefore, the approach to choosing technologies is quite different.[37]

The Eclipse Foundation has published a specification for developing microservices, Eclipse MicroProfile.[38][39]

Service mesh

In a service mesh, each service instance is paired with an instance of a reverse proxy server, called a service proxy, sidecar proxy, or sidecar. The service instance and sidecar proxy share a container, and the containers are managed by a container orchestration tool such as Kubernetes, Docker Swarm, or DC/OS. The service proxies are responsible for communication with other service instances and can support capabilities such as service (instance) discovery, load balancing, authentication and authorization, secure communications, and others.

See also

- Conway's law
- Cross-cutting concern
- Data mesh, a domain-oriented data architecture
- DevOps
- Fallacies of distributed computing
- GraphQL
- gRPC
- Interface description language (IDL)
- Representational state transfer (REST)
- Service-oriented architecture (SOA)
- Microfrontend
- Unix philosophy
- Self-contained system (software)
- Serverless computing
- Web-oriented architecture (WOA)

References

1. Fowler, Martin (2002). *Patterns of Enterprise Application Architecture*. Addison-Wesley Professional. ISBN 978-0321127426.
2. Newman, Sam (2015-02-20). *Building Microservices*. O'Reilly Media. ISBN 978-1491950357.
3. Wolff, Eberhard (2016-10-12). *Microservices: Flexible Software Architectures*. Addison-Wesley. ISBN 978-0134602417.
4. Nadareishvili, I., Mitra, R., McLarty, M., Amundsen, M., Microservice Architecture: Aligning Principles, Practices, and Culture, O'Reilly 2016
5. Martin Fowler (28 August 2014). "Microservice Prerequisites" (https://martinfowler.com/bliki/MicroservicePrerequisites.html). Archived (https://web.archive.org/web/20231003075551/https://martinfowler.com/bliki/MicroservicePrerequisites.html) from the original on Oct 3, 2023.
6. Richardson, Chris (November 2018). *Microservice Patterns*. Manning Publications. 1.4.1 Scale cube and microservices. ISBN 9781617294549.
7. Mendonca, Nabor C.; Jamshidi, Pooyan; Garlan, David; Pahl, Claus (2019-10-16). "Developing Self-Adaptive Microservice Systems: Challenges and Directions". *IEEE Software*. **38** (2): 70–79. arXiv:1910.07660 (https://arxiv.org/abs/1910.07660). doi:10.1109/MS.2019.2955937 (https://doi.org/10.1109%2FMS.2019.2955937). S2CID 204744007 (https://api.semanticscholar.org/CorpusID:204744007).
8. Richardson, Chris (2019). *Microservices patterns: with examples in Java*. Shelter Island, NY: Manning Publications. ISBN 978-1-61729-454-9.
9. Christudas, Binildas (2019). *Practical Microservices Architectural Patterns: Event-Based Java Microservices with Spring Boot and Spring Cloud*. Berkeley, CA: Apress L. P. ISBN 978-1-4842-4501-9.
10. Russell, Perry; Rodgers, Peter; Sellman, Royston (2004). "Architecture and Design of an XML Application Platform" (http://www.hpl.hp.com/techreports/2004/HPL-2004-23.html). *HP Technical Reports*. p. 62. Retrieved 20 August 2015.
11. Rodgers, Peter (Feb 15, 2005). "Service-Oriented Development on NetKernel- Patterns, Processes & Products to Reduce System Complexity" (https://web.archive.org/web/20180520124343/http://www.cloudcomputingexpo.com/node/80883). *CloudComputingExpo*. SYS-CON Media. Archived from the original (http://www.cloudcomputingexpo.com/node/80883) on 20 May 2018. Retrieved 19 August 2015.
12. Richards, Mark; Ford, Neal (2020). *Fundamentals of software architecture: an engineering approach* (First ed.). O'Reilly. ISBN 9781492043454.
13. Richards, Mark. *Microservices AntiPatterns and Pitfalls*. O'Reilly.
14. *Building Microservices by Sam Newman*. ISBN 978-1492034025.
15. Chen, Lianping (2018). *Microservices: Architecting for Continuous Delivery and DevOps* (https://www.researchgate.net/publication/323944215). The IEEE International Conference on Software Architecture (ICSA 2018) (http://icsa-conferences.org/2018/). IEEE.
16. Yousif, Mazin (2016). "Microservices". *IEEE Cloud Computing*. **3** (5): 4–5. doi:10.1109/MCC.2016.101 (https://doi.org/10.1109%2FMCC.2016.101).
17. Dragoni, Nicola; Lanese, Ivan; Larsen, Stephan Thordal; Mazzara, Manuel; Mustafin, Ruslan; Safina, Larisa (2017). "Microservices: How to Make Your Application Scale" (https://hal.inria.fr/hal-01636132/file/microservices-make-application.pdf) (PDF). *Perspectives of System Informatics*. Lecture Notes in Computer Science. Vol. 10742. pp. 95–104. arXiv:1702.07149 (https://arxiv.org/abs/1702.07149). Bibcode:2017arXiv170207149D (https://ui.adsabs.harvard.edu/abs/2017arXiv170207149D). doi:10.1007/978-3-319-74313-4_8 (https://doi.org/10.1007%2F978-3-319-74313-4_8). ISBN 978-3-319-74312-7. S2CID 1643730 (https://api.semanticscholar.org/CorpusID:1643730).

18. Newman, Sam (2015). *Building Microservices*. O'Reilly. ISBN 978-1491950357.
19. Wolff, Eberhard (2016). *Microservices: Flexible Software Architecture*. Addison Wesley. ISBN 978-0134602417.
20. Knoche, Holger; Hasselbring, Wilhelm (2019). "Drivers and Barriers for Microservice Adoption – A Survey among Professionals in Germany" (https://www.researchgate.net/publication/330442182). *Enterprise Modelling and Information Systems Architectures*. **14**: 1:1–35–1:1–35. doi:10.18417/emisa.14.1 (https://doi.org/10.18417%2Femisa.14.1).
21. Taibi, Davide; Lenarduzzi, Valentina; Pahl, Claus; Janes, Andrea (2017). "Microservices in agile software development: A workshop-based study into issues, advantages, and disadvantages" (https://www.researchgate.net/publication/319131505). *Proceedings of the XP2017 Scientific Workshops*. pp. 1–5. doi:10.1145/3120459.3120483 (https://doi.org/10.1145%2F3120459.3120483). ISBN 978-1-4503-5264-2. S2CID 28134110 (https://api.semanticscholar.org/CorpusID:28134110).
22. Richardson, Chris. "Microservice architecture pattern" (http://microservices.io/patterns/microservices.html). *microservices.io*. Retrieved 2017-03-19.
23. Chen, Lianping; Ali Babar, Muhammad (2014). "Towards an Evidence-Based Understanding of Emergence of Architecture through Continuous Refactoring in Agile Software Development". *Proceedings Working IEEE/IFIP Conference on Software Architecture 2014 WICSA 2014*. The 11th Working IEEE/IFIP Conference on Software Architecture(WICSA 2014) (https://web.archive.org/web/20140730053454/http://wicsa2014.org/). IEEE. doi:10.1109/WICSA.2014.45 (https://doi.org/10.1109%2FWICSA.2014.45).
24. Balalaie, Armin; Heydarnoori, Abbas; Jamshidi, Pooyan (May 2016). "Microservices Architecture Enables DevOps: Migration to a Cloud-Native Architecture" (http://spiral.imperial.ac.uk/bitstream/10044/1/40557/8/SO_SWSI-2015-10-0149.R1_Balalaie.pdf) (PDF). *IEEE Software*. **33** (3): 42–52. doi:10.1109/ms.2016.64 (https://doi.org/10.1109%2Fms.2016.64). hdl:10044/1/40557 (https://hdl.handle.net/10044%2F1%2F40557). ISSN 0740-7459 (https://search.worldcat.org/issn/0740-7459). S2CID 18802650 (https://api.semanticscholar.org/CorpusID:18802650).
25. Stenberg, Jan (11 August 2014). "Experiences from Failing with Microservices" (http://www.infoq.com/news/2014/08/failing-microservices).
26. Martin Fowler. "Microservices" (http://martinfowler.com/articles/microservices.html). Archived (https://web.archive.org/web/20180214171522/https://martinfowler.com/articles/microservices.html) from the original on 14 February 2018.
27. Calandra, Mariano (7 April 2021). "Why unit testing is not enough when it comes to microservices" (https://medium.com/swlh/why-unit-testing-is-not-enough-when-it-comes-to-microservices-c3b0dde14174).
28. Lanza, Michele; Ducasse, Stéphane (2002). "Understanding Software Evolution using a Combination of Software Visualization and Software Metrics" (https://web.archive.org/web/20210227104157/https://rmod.inria.fr/archives/papers/Lanz02aEvolutionMatrix.pdf) (PDF). *Proceedings of LMO 2002 (Langages et Modèles à Objets)*: 135–149. Archived from the original (https://rmod.inria.fr/archives/papers/Lanz02aEvolutionMatrix.pdf) (PDF) on Feb 27, 2021.
29. Richardson, Chris (November 2018). "Chapter 4. Managing transactions with sagas". *Microservice Patterns*. Manning Publications. ISBN 978-1-61729454-9.
30. Devoxx (Aug 30, 2017). "10 Tips for failing badly at Microservices by David Schmitz" (https://web.archive.org/web/20210422161909/https://www.youtube.com/watch?v=X0tjziAQfNQ). *YouTube*. Archived from the original (https://www.youtube.com/watch?v=X0tjziAQfNQ) on Apr 22, 2021.
31. Löwy, Juval (2019). *Righting Software 1st ed*. Addison-Wesley Professional. pp. 73–75. ISBN 978-0136524038.

32. Pautasso, Cesare (2017). "Microservices in Practice, Part 2: Service Integration and Sustainability". *IEEE Software*. **34** (2): 97–104. doi:10.1109/MS.2017.56 (https://doi.org/10.1109%2FMS.2017.56). S2CID 30256045 (https://api.semanticscholar.org/CorpusID:30256045).
33. Pautasso, Cesare (2018). "Consistent Disaster Recovery for Microservices: the BAC Theorem". *IEEE Cloud Computing*. **5** (1): 49–59. doi:10.1109/MCC.2018.011791714 (https://doi.org/10.1109%2FMCC.2018.011791714). S2CID 4560021 (https://api.semanticscholar.org/CorpusID:4560021).
34. "Developing Microservices for PaaS with Spring and Cloud Foundry" (http://www.infoq.com/presentations/microservices-pass-spring-cloud-foundry).
35. Fowler, Martin. "Microservice Trade-Offs" (https://www.martinfowler.com/articles/microservice-trade-offs.html#ops).
36. "BRASS Building Resource Adaptive Software Systems". U.S. Government. DARPA. April 7, 2015. "Access to system components and the interfaces between clients and their applications, however, are mediated via a number of often unrelated mechanisms, including informally documented application programming interfaces (APIs), idiosyncratic foreign function interfaces, complex ill-understood model definitions, or *ad hoc* data formats. These mechanisms usually provide only partial and incomplete understanding of the semantics of the components themselves. In the presence of such complexity, it is not surprising that applications typically bake-in many assumptions about the expected behavior of the ecosystem they interact with".
37. Wolff, Eberhard (2018-04-15). *Microservices - A Practical Guide* (http://practical-microservices.com). CreateSpace Independent Publishing Platform. ISBN 978-1717075901.
38. Swart, Stephanie (14 December 2016). "Eclipse MicroProfile" (https://projects.eclipse.org/projects/technology.microprofile). *projects.eclipse.org*.
39. "MicroProfile" (https://microprofile.io/). *MicroProfile*. Retrieved 2021-04-11.

Further reading

- "Special theme issue on microservices" (https://ieeexplore.ieee.org/xpl/tocresult.jsp?isnumber=8354413). *IEEE Software* . **35** (3). May–June 2018.
- I. Nadareishvili et al., Microservices Architecture – Aligning Principles, Practices and Culture (https://www.ca.com/content/dam/ca/us/files/ebook/microservice-architecture-aligning-principles-practices-and-culture.pdf), O'Reilly, 2016, ISBN 978-1-491-95979-4
- S. Newman, Building Microservices – Designing Fine-Grained Systems, O'Reilly, 2015 ISBN 978-1491950357
- Wijesuriya, Viraj Brian (2016-08-29) *Microservice Architecture, Lecture Notes (https://www.slideshare.net/tyrantbrian/microservice-architecture-65505794)* - University of Colombo School of Computing, Sri Lanka
- Christudas Binildas (June 27, 2019). Practical Microservices Architectural Patterns: Event-Based Java Microservices with Spring Boot and Spring Cloud. Apress. ISBN 978-1484245002.

Retrieved from "https://en.wikipedia.org/w/index.php?title=Microservices&oldid=1336297142"